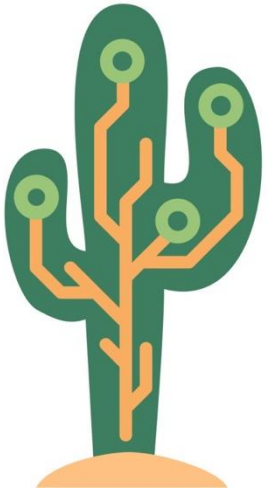
Ira A. Fulton Schools of Engineering
Arizona State University

Advanced Hardware and Systems for Machine Learning

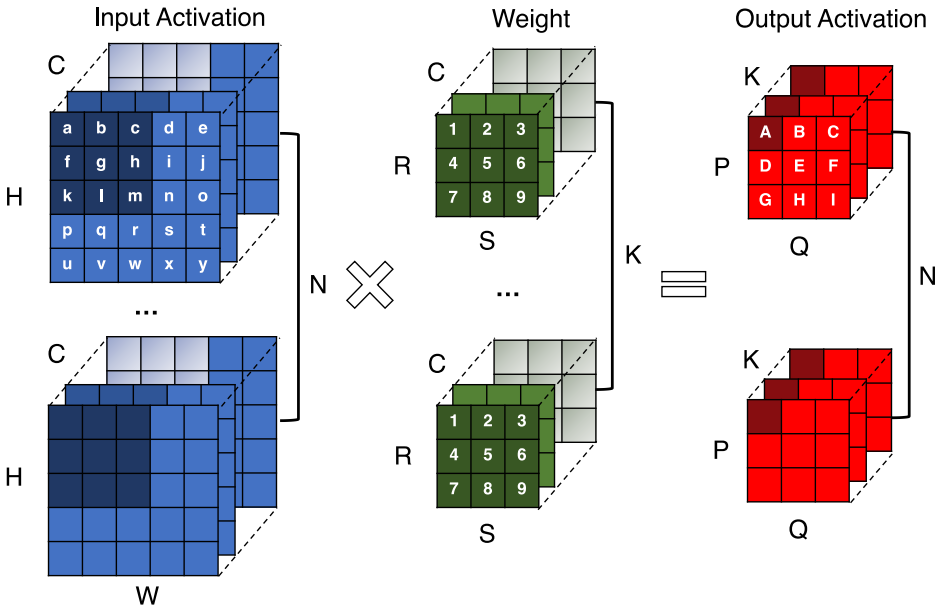
Lecture 10: Data Flow



Jeff (Jun) Zhang
Assistant Professor
School of ECEE, ASU
jeffzhang@asu.edu



Convolution Loop Nest

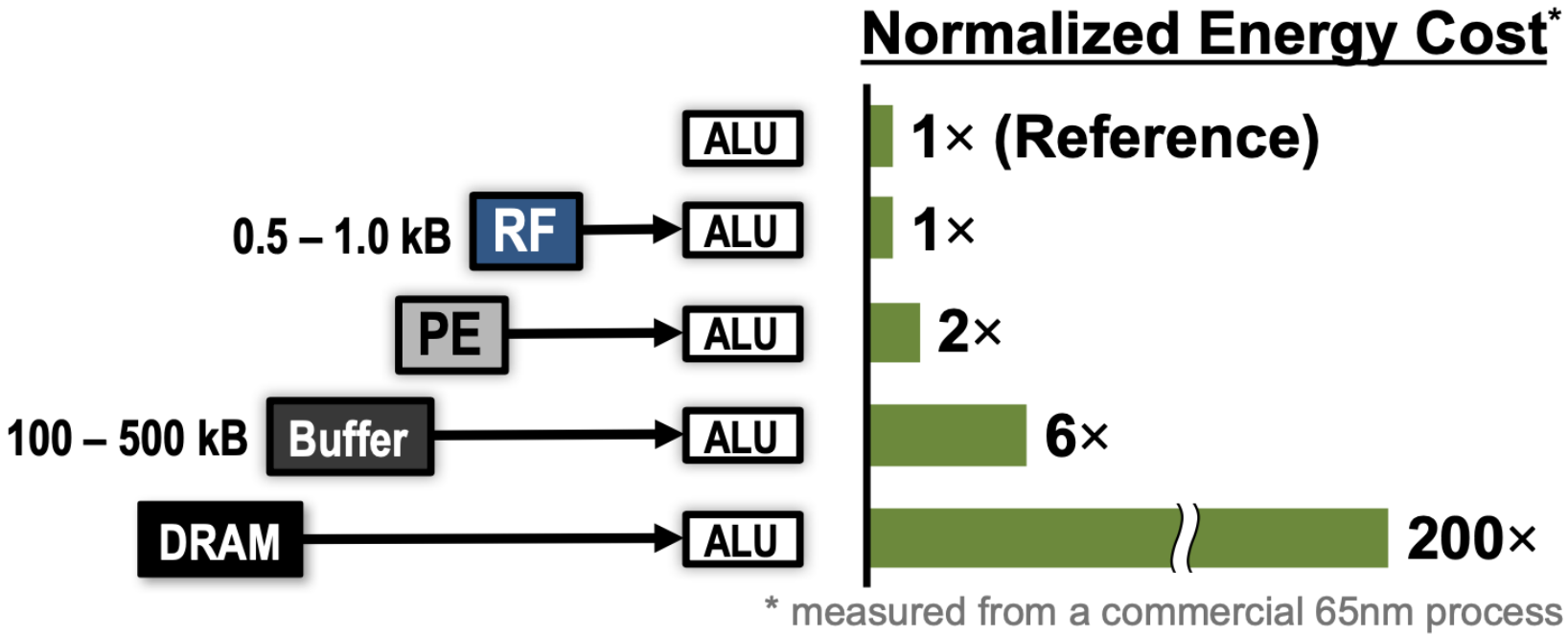



```
for (n=0; n<N; n++) {
  for (k=0; k<K; k++) {
    for (p=0; p<P; p++) {
      for (q=0; q<Q; q++) {
        OA[n][k][p][q]= 0;
        for (r=0; r<R; r++) {
          for (s=0; s<S; s++) {
            for (c=0; c<C; c++) {
              h = p * stride - pad + r;
              w = q * stride - pad + s;
              OA[n][k][p][q] +=
                                  IA[n][c][h][w]
                              * W[k][c][r][s];
            }
          }
        }
        OA[n][k][p][q]= Activation(OA[n][k][p][q]);
      }
    }
  }
}
```

for each output activation

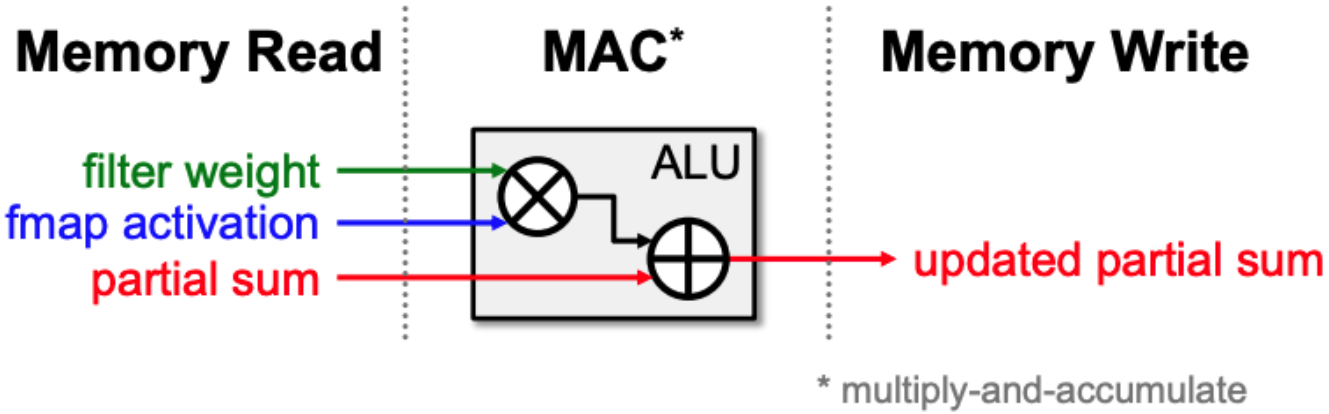
convolution window

Locality: Data Movement Cost

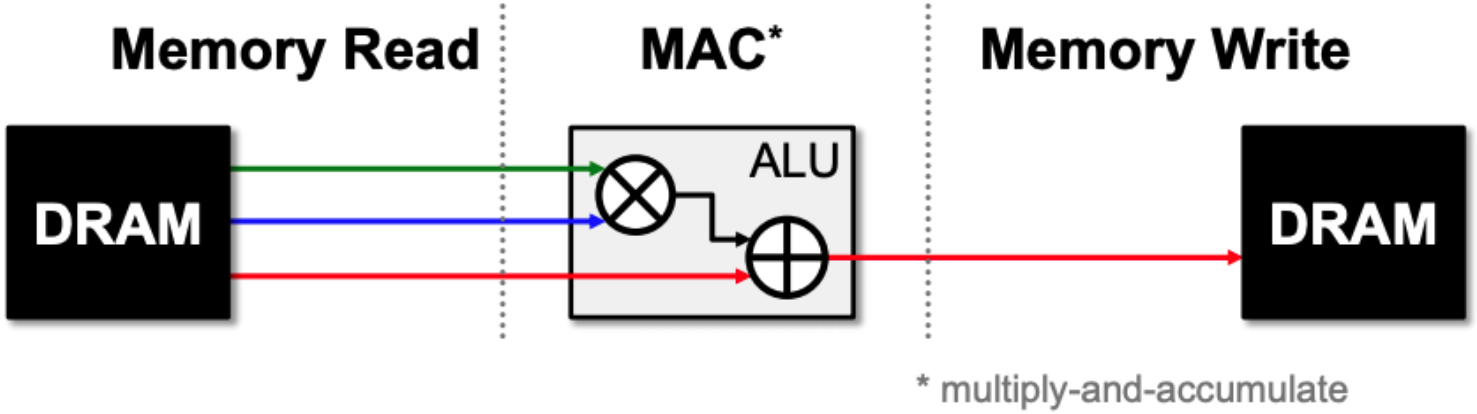


Eyeriss Tutorial

Locality: Memory Access in DNNs



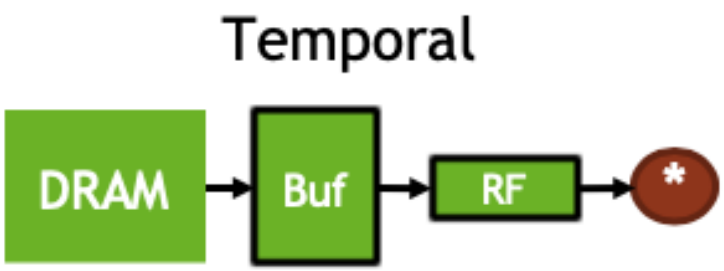
Locality: Memory Access in DNNs

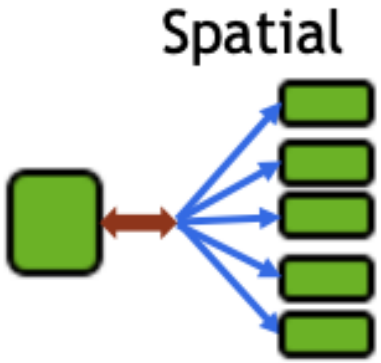


- Worst case: all memory R/W are DRAM accesses
- Example: AlexNet [NIPS 2012] has 724M MACs -> 2896M DRAM accesses required.

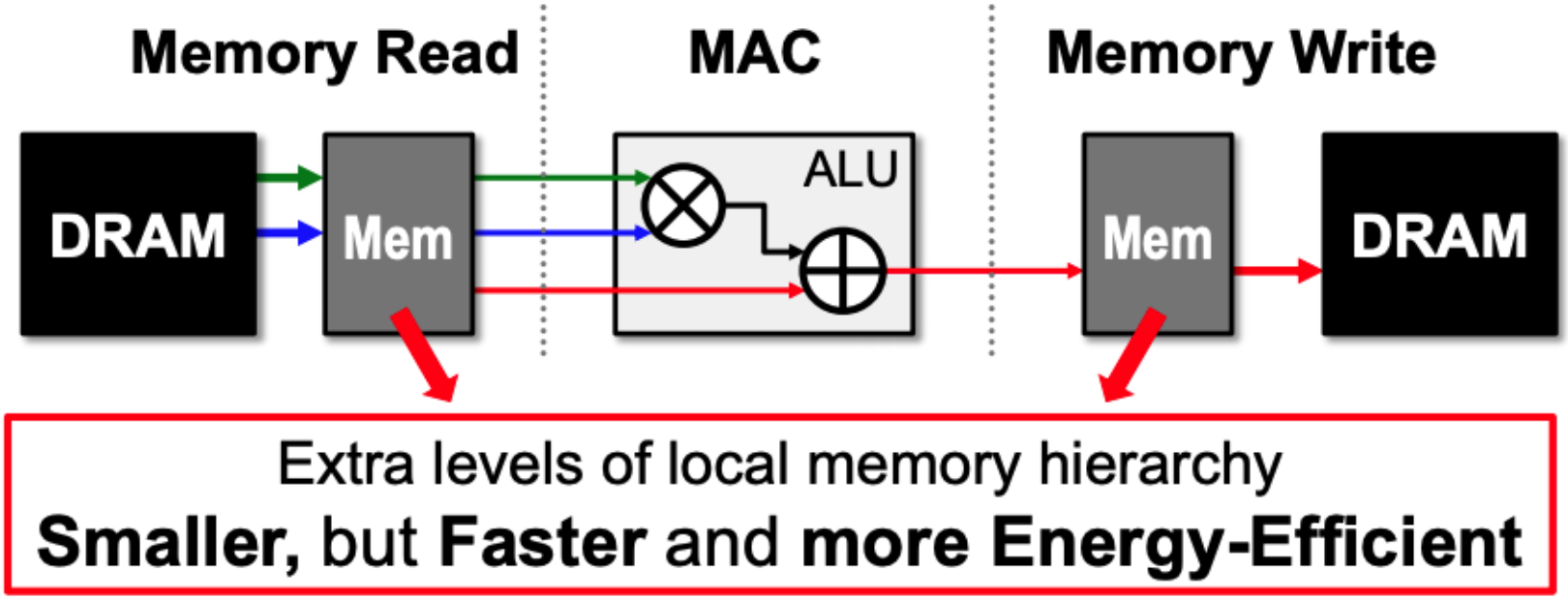
Locality: Temporal and Spatial Reuse

- Temporal reuse: the same data is used more than once over time by the same consumer.
- Spatial reuse: the same data is used by more than one consumer at different spatial locations of the hardware.





Locality: Memory Hierarchy

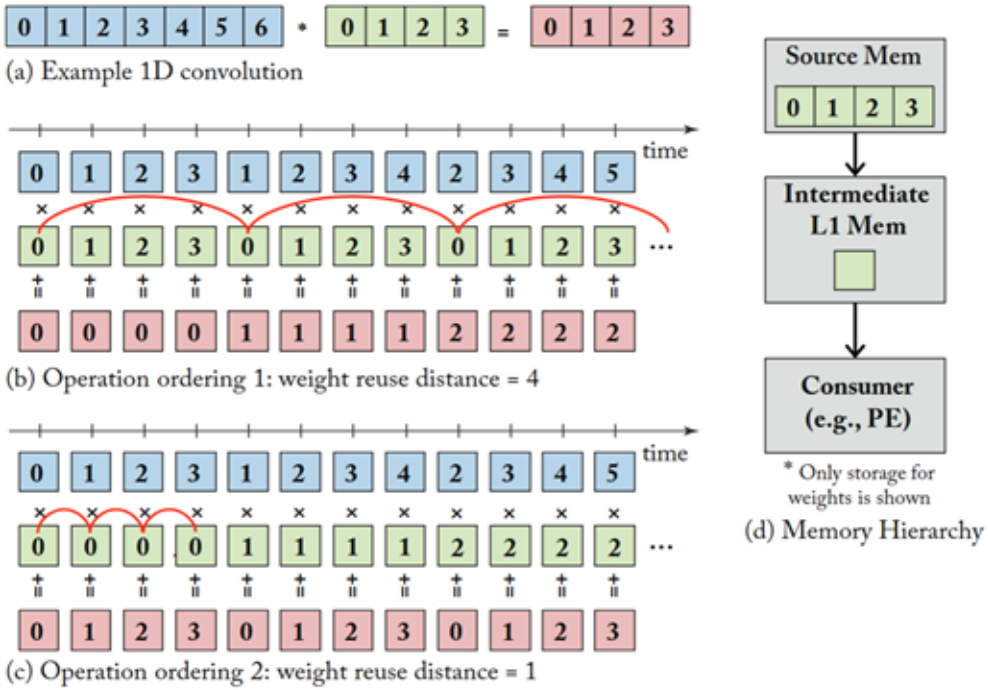


- Temporal reuse: the same data is used more than once over time by the same consumer.

Temporal Reuse Example

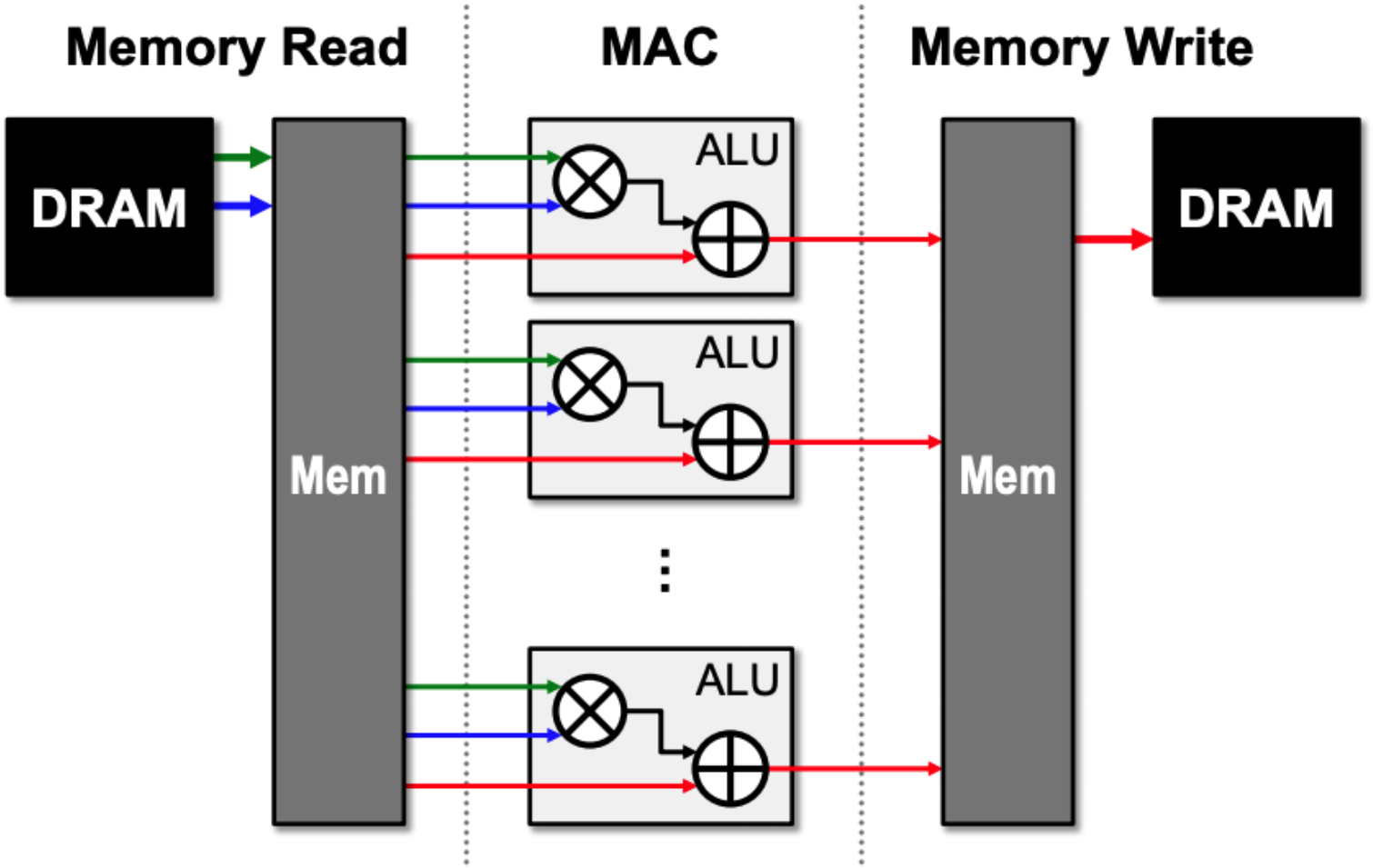
Reuse Distance

- Number of memory accesses
- Size of your on-chip memory
- Smaller is better



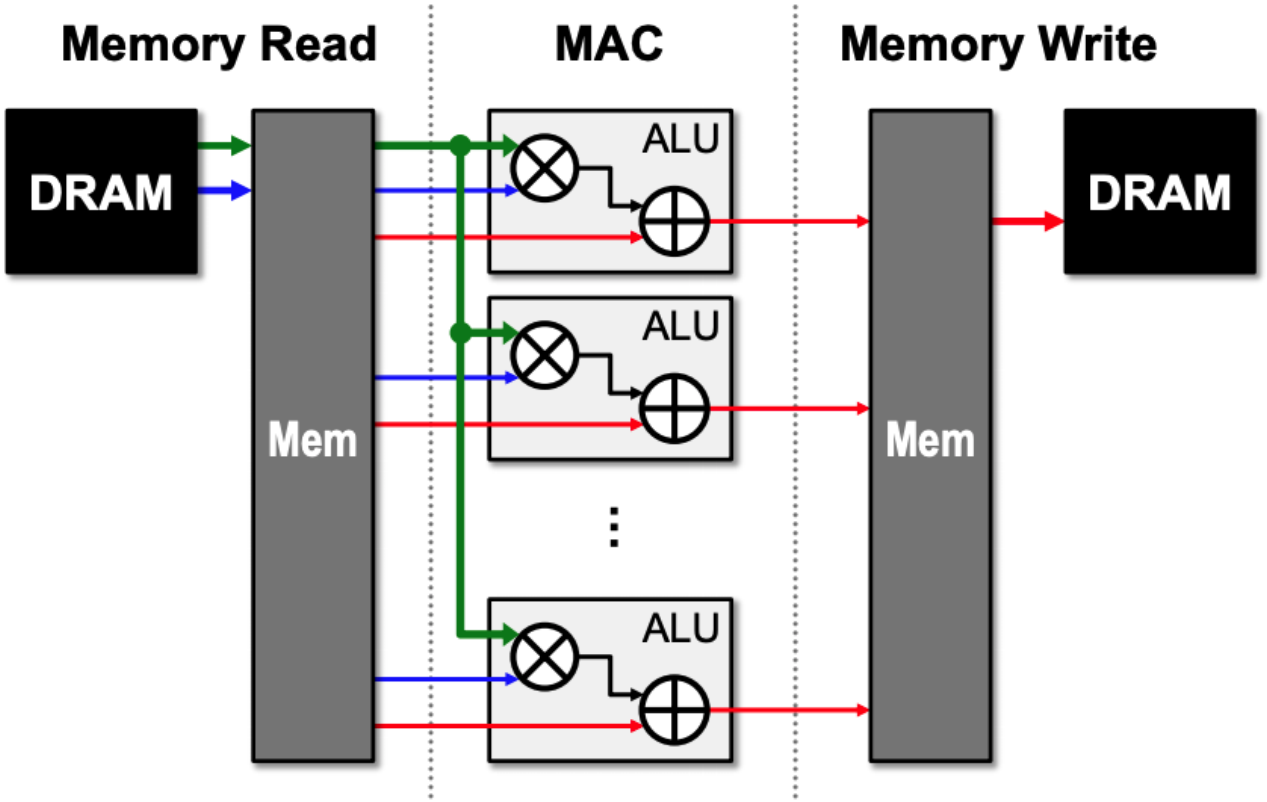
Source: V. Sze et al. "Efficient Processing of Deep Neural Networks" 2020

Parallelism: Parallel MAC Units

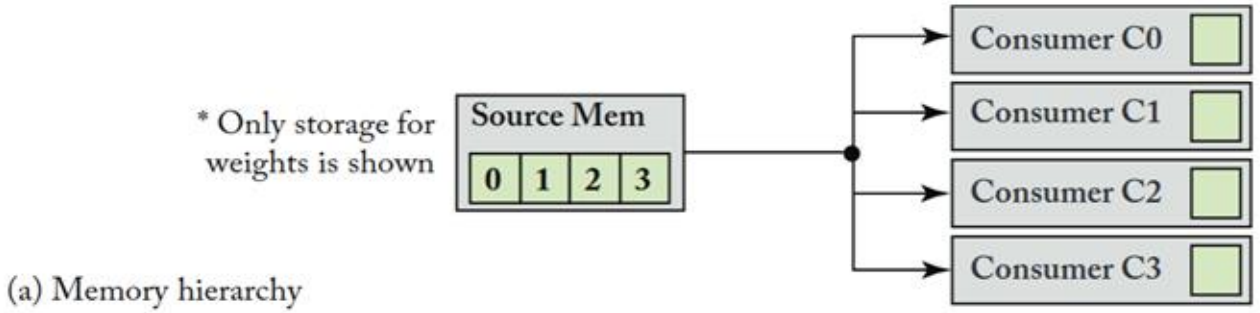


Parallelism: Spatial Data Reuse

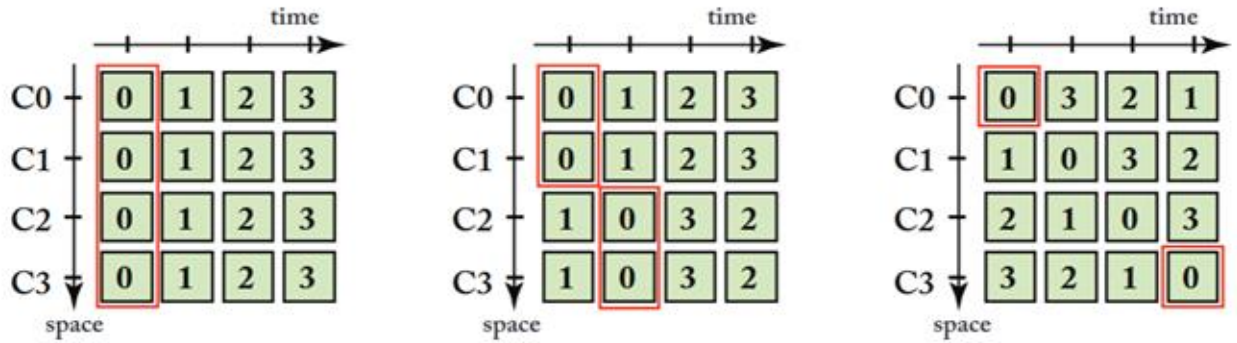
- Spatial reuse: the same data is used by more than one consumer at different spatial locations of the hardware.



Spatial Reuse Example



(a) Memory hierarchy



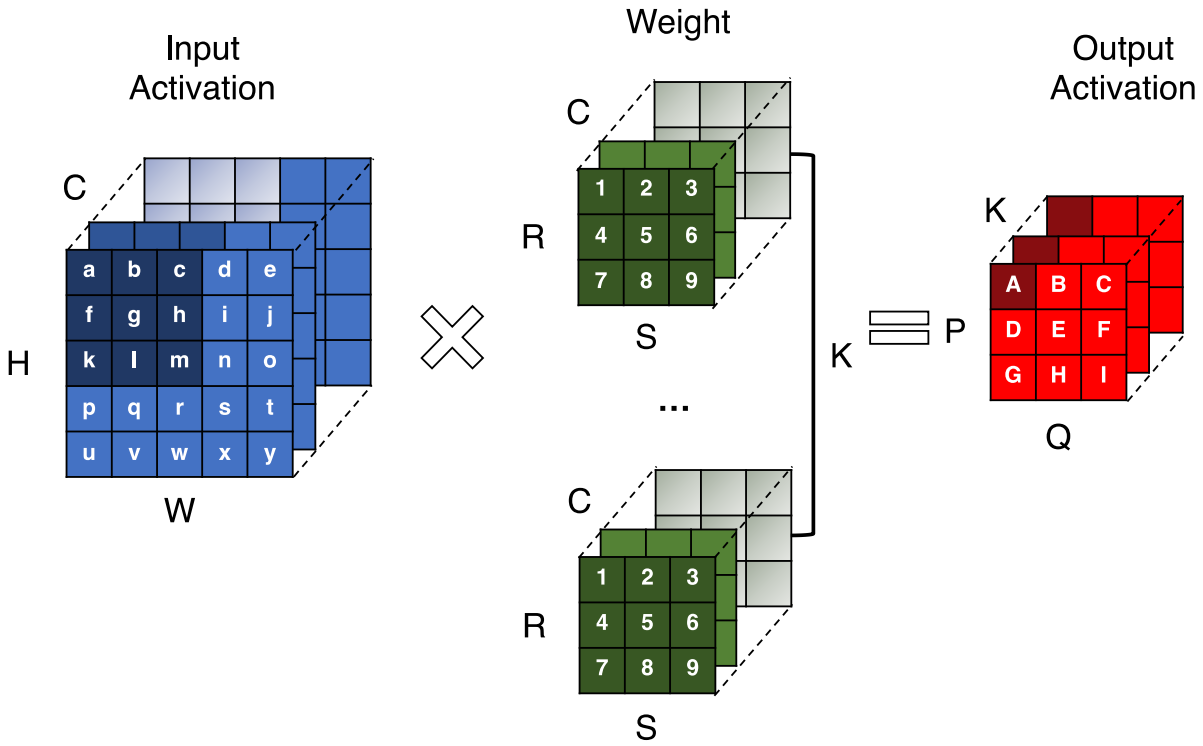
(b) Operation ordering 1 (c) Operation ordering 2 (d) Operation ordering 3

Why a smaller reuse distance with spatial reuse?

- Fewer memory read ports
  - Less area
- Fewer reads
  - Less power

Source: V. Sze et al. "Efficient Processing of Deep Neural Networks" 2020

Data Reuse in DNN

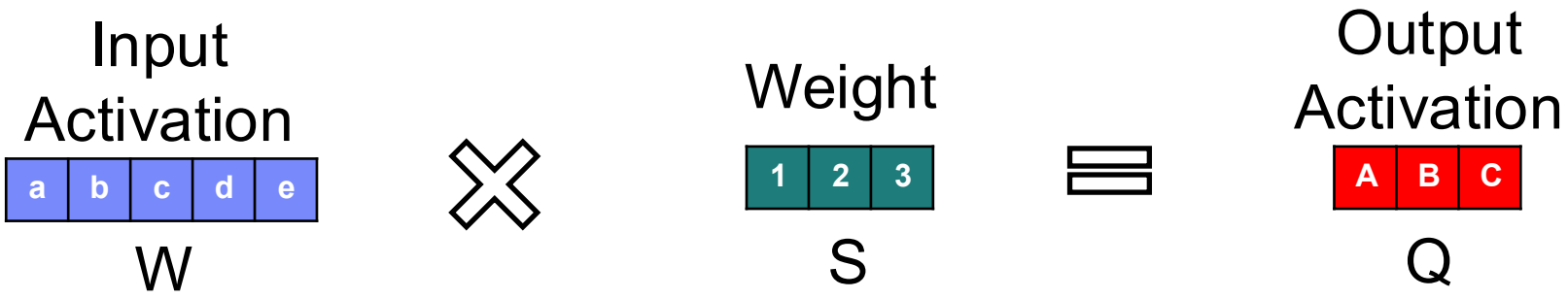


- **Total # of MACs:**
  - RSCPQK
- **Input Activation**
  - Size: HWC
  - Reuse:~RSK
- **Weight**
  - Size: RSCK
  - Reuse: PQ
- **Output Activation**
  - Size: PQK
  - Reuse: RSC

Dataflow

- **Defines the execution order of the DNN operations in hardware.**
  - Computation order
  - Data movement order
- **Loop nest is a compact way to describe the execution order, i.e., dataflow, supported in hardware.**
  - `for`: temporal for, describes the temporal execution order.
  - `spatial_for`: describes parallel execution.

1D Convolution Example



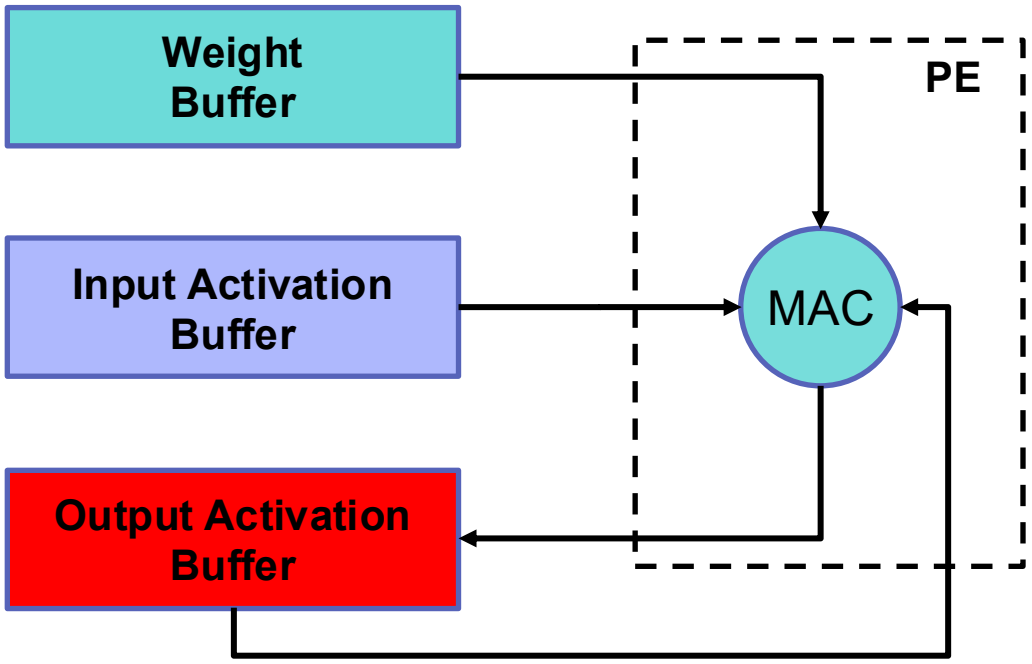

```
for(q=0; q<Q; q++){
  for (s=0; s<S; s++){
    OA[q] += IA[q+s] * W[s];
  }
}
```

Output Stationary (OS) Dataflow

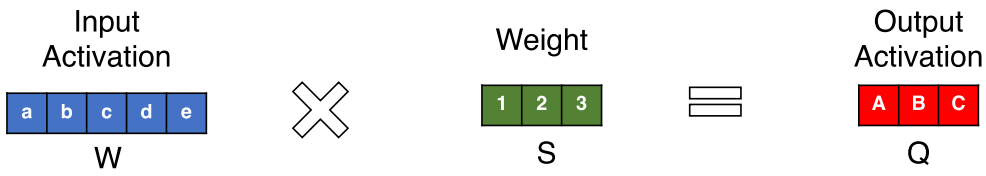
```
for (s=0; s<S; s++){
  for(q=0; q<Q; q++){
    OA[q] += IA[q+s] * W[s];
  }
}
```

Weight Stationary (WS) Dataflow

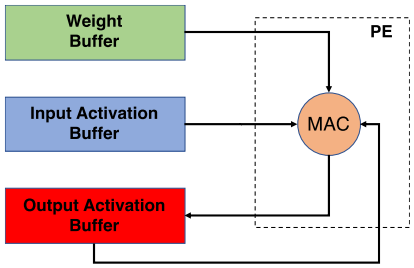
Single Processing Element (PE) Setup

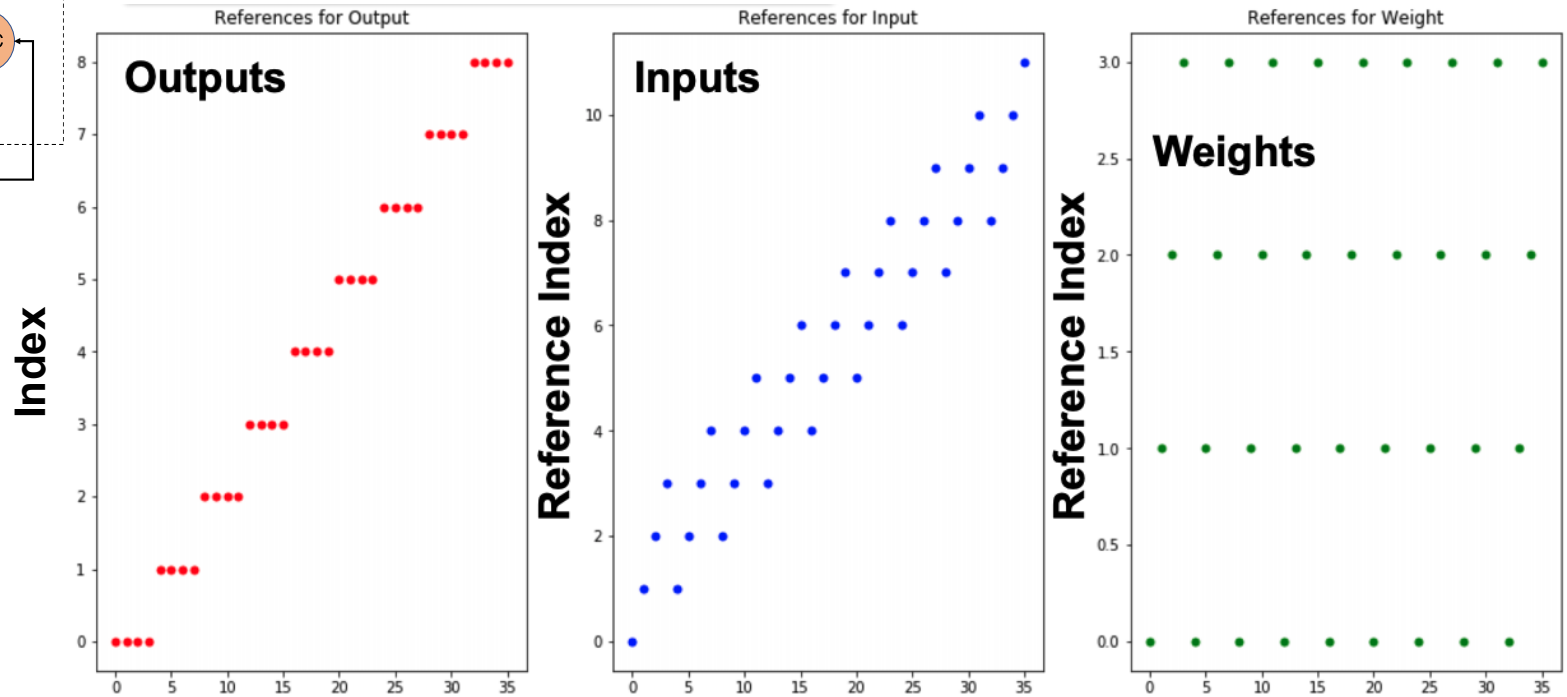


Buffer Access Pattern (OS)



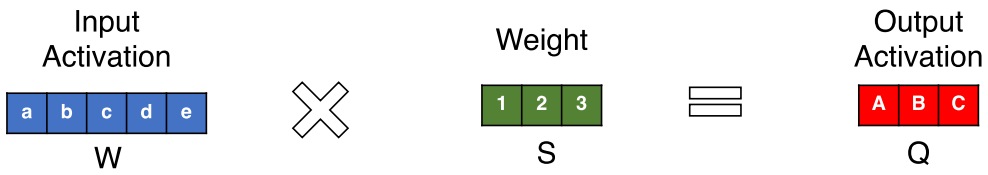

```
for(q=0; q<Q; q++){ // Q =9
  for (s=0; s<S; s++){ // S=4
    OA[q] += IA[q+s] * W[s];
  }
}
```



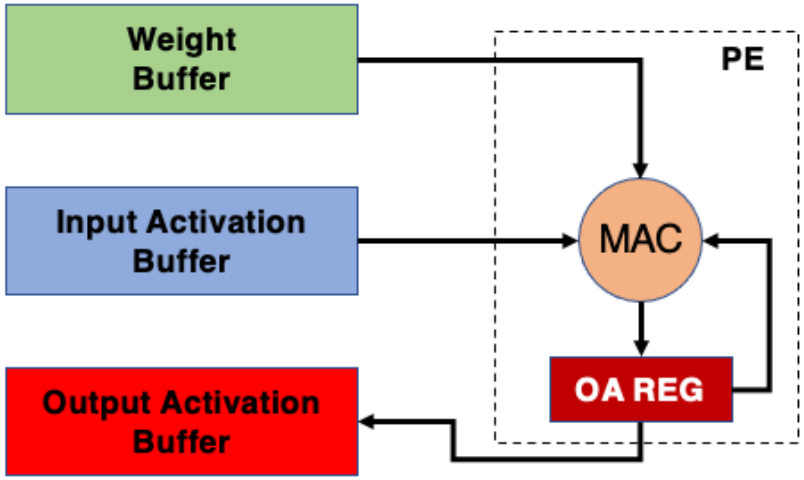


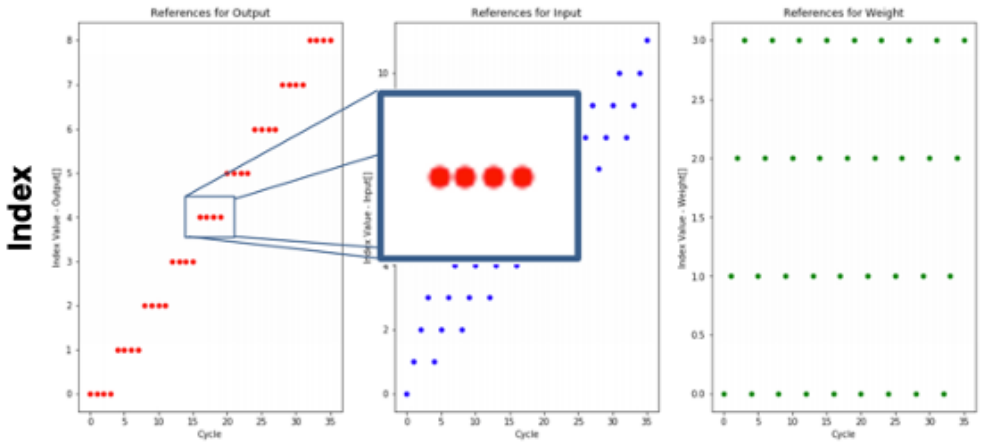
Eyeriss Tutorial

Buffer Access Pattern (OS)




```
for(q=0; q<Q; q++){ // Q =9
  for (s=0; s<S; s++){ // S=4
    OA[q] += IA[q+s] * W[s];
  }
}
```

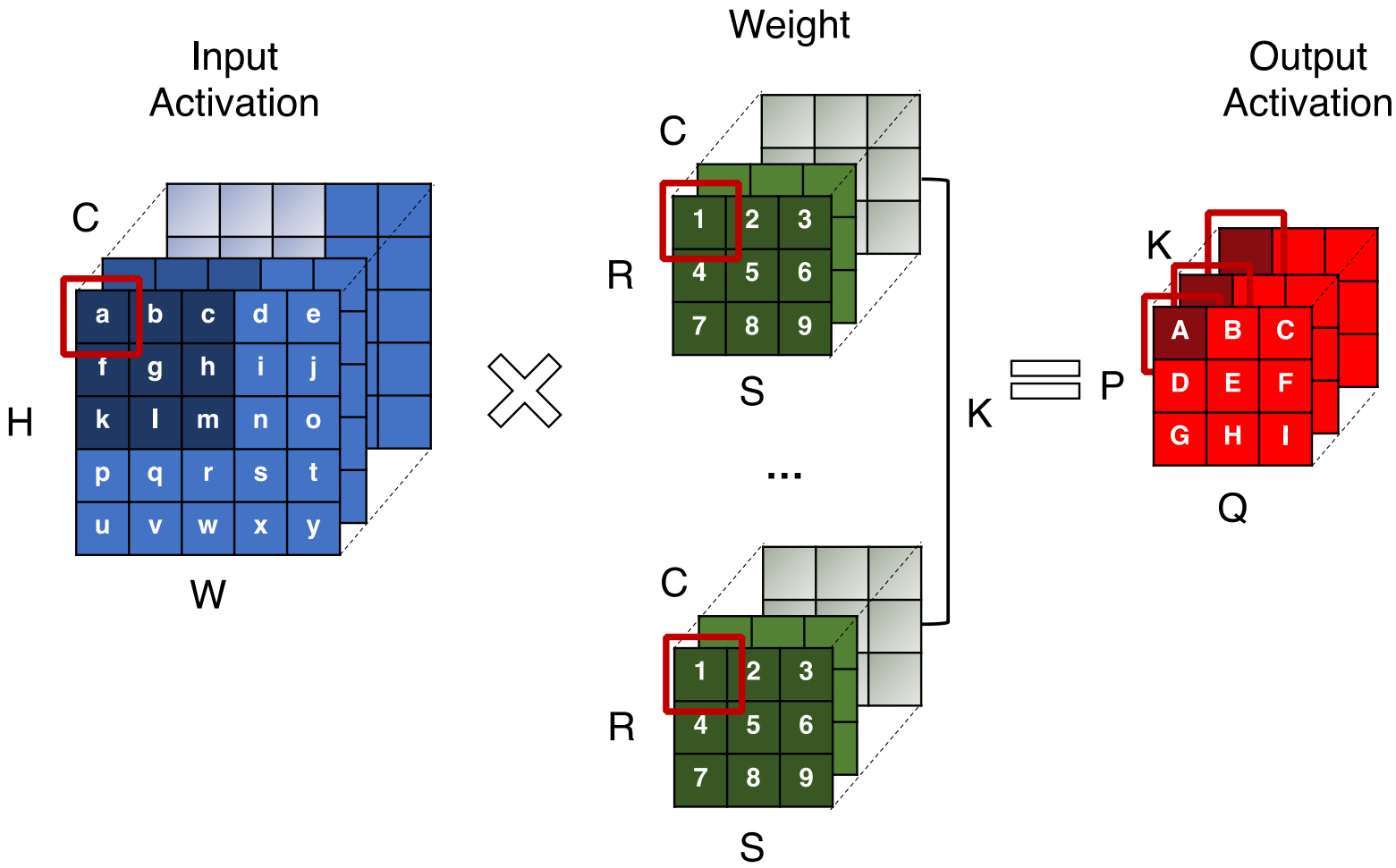




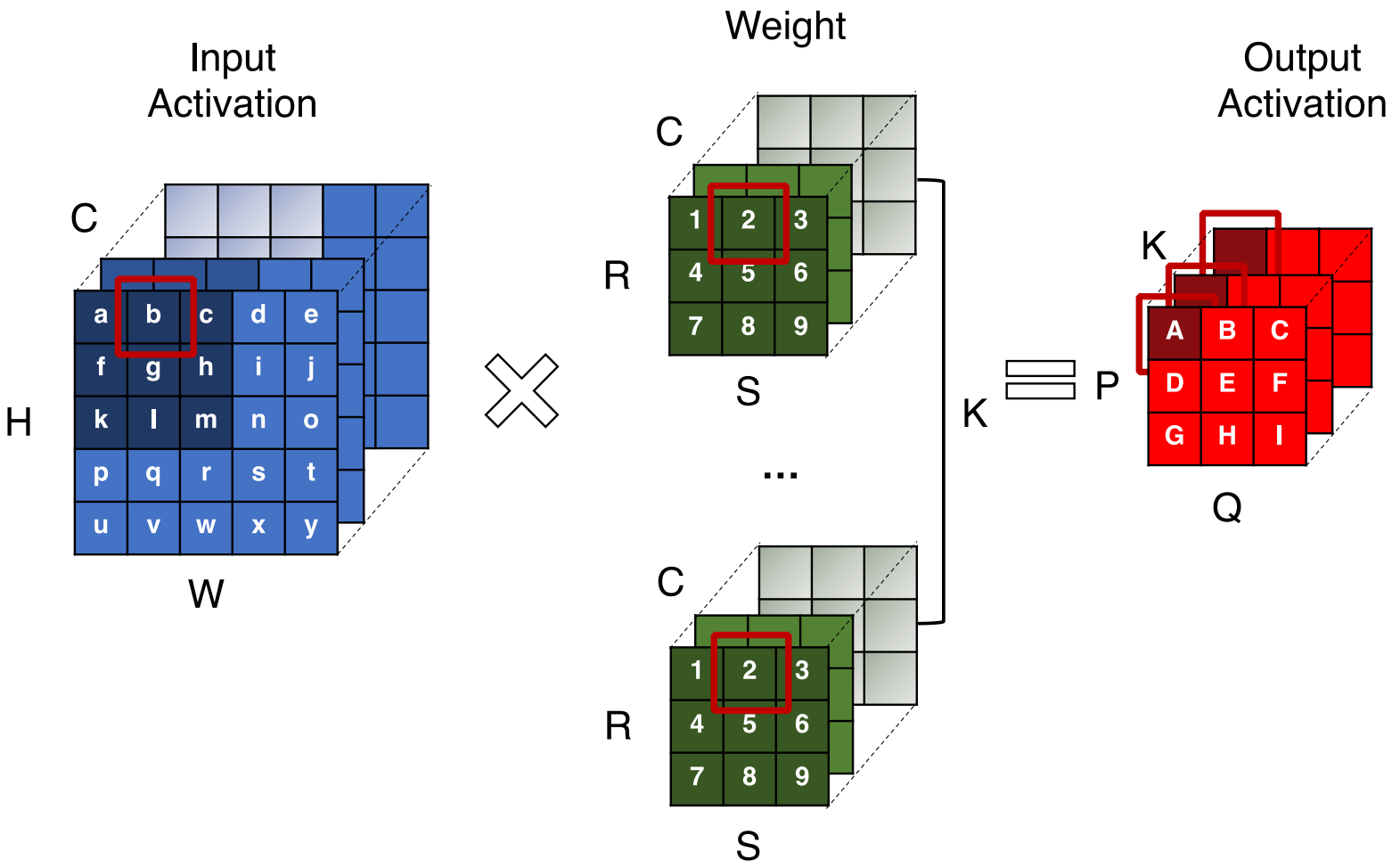
Observation: Single output is reused S times.

Eyeriss Tutorial

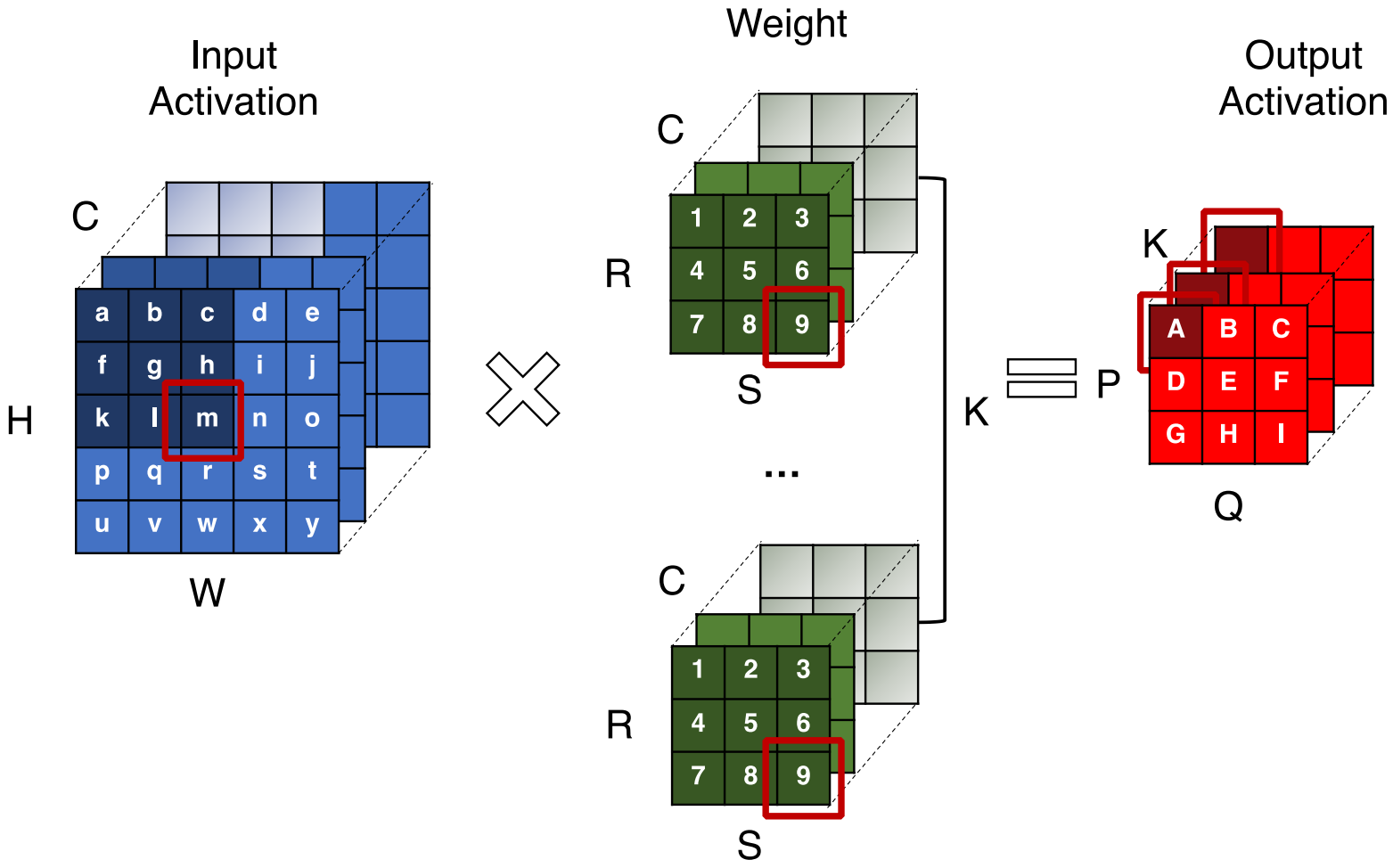
Output Stationary Dataflow



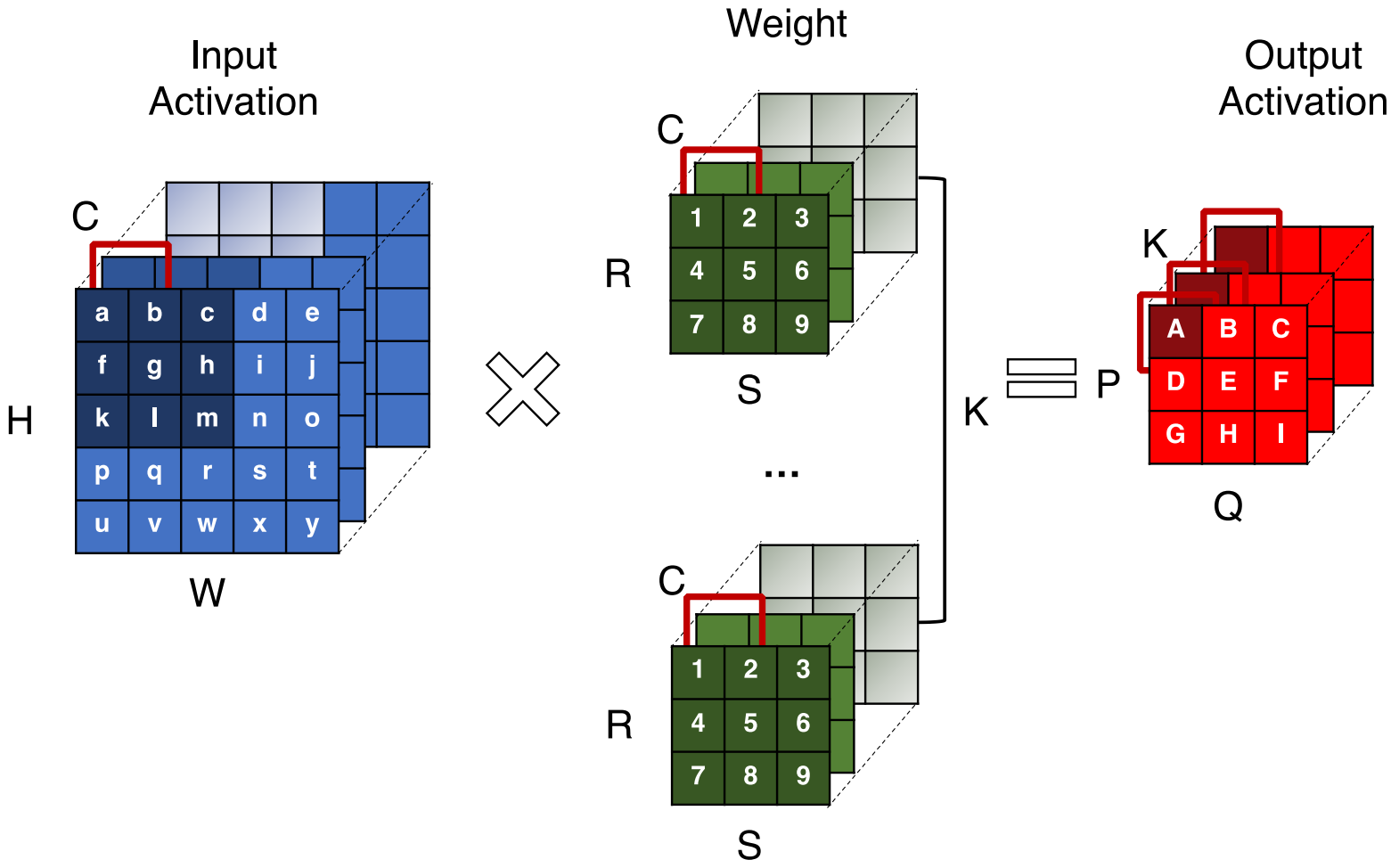
Output Stationary Dataflow



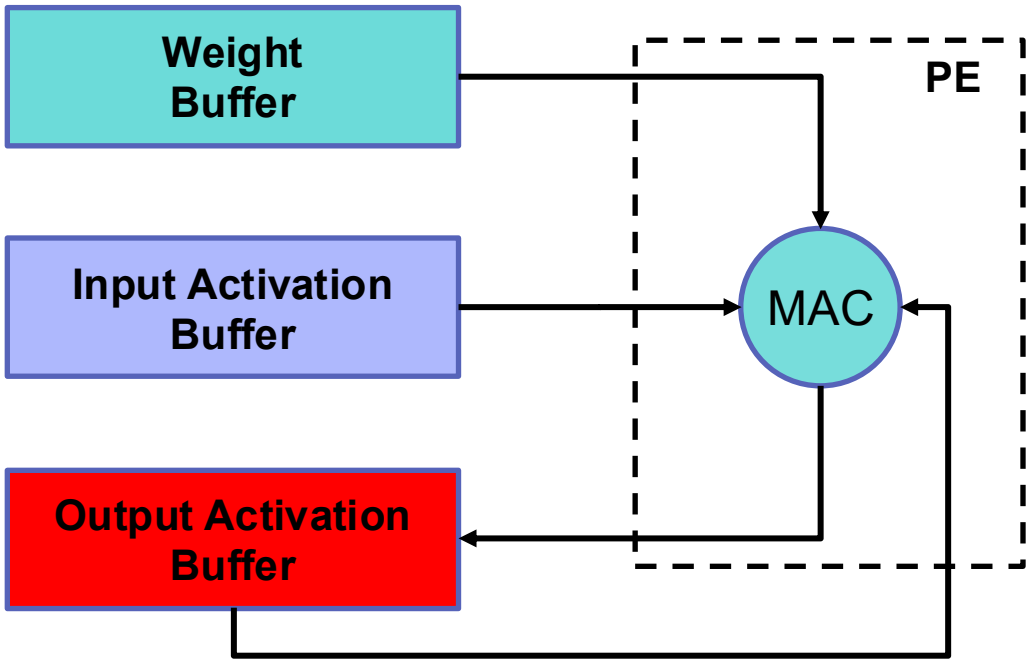
Output Stationary Dataflow



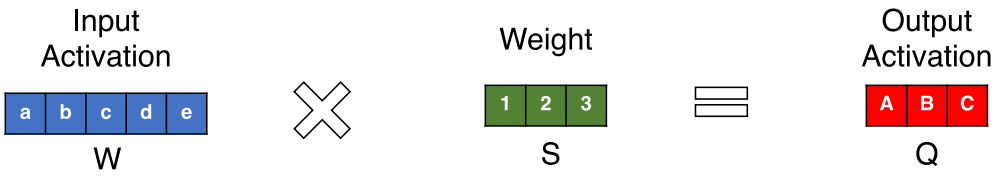
Output Stationary Dataflow



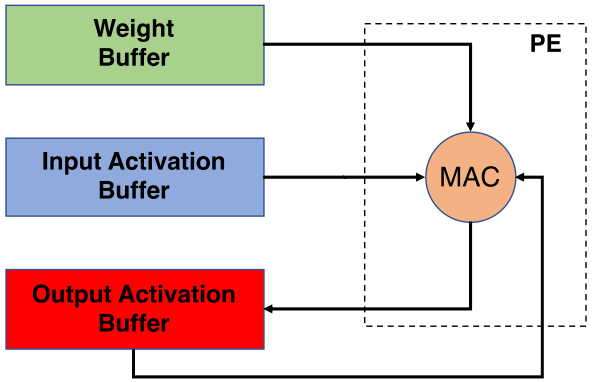
Single Processing Element (PE) Setup



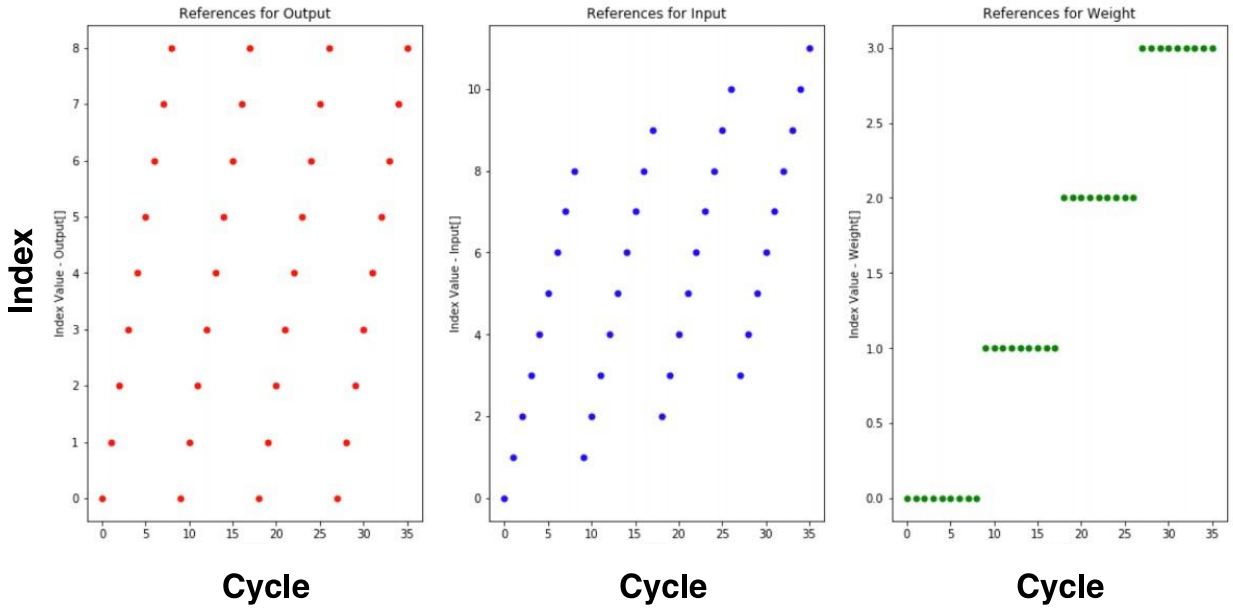
Buffer Access Pattern (WS)



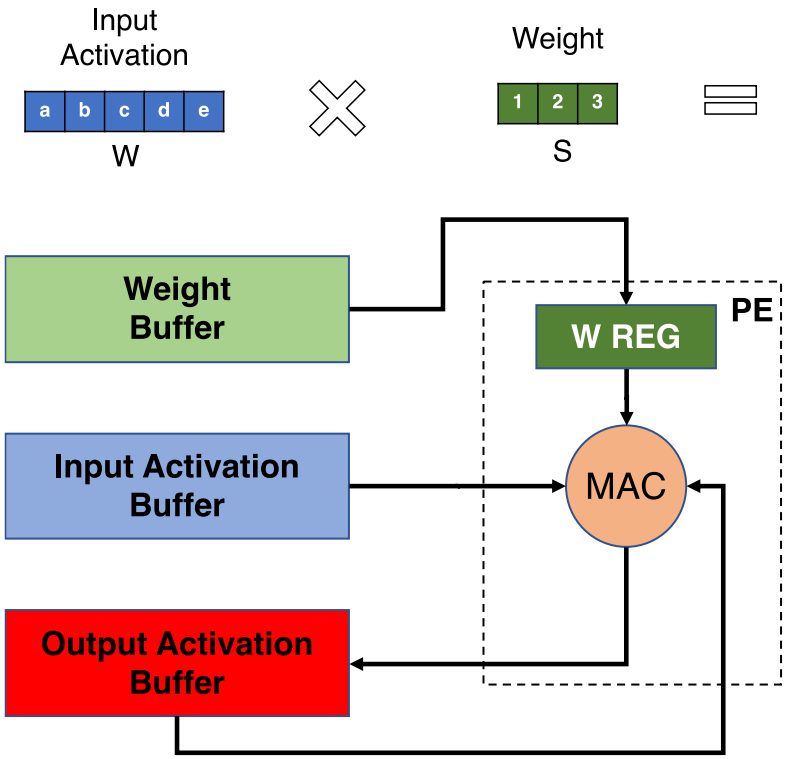

```
for (s=0; s<S; s++){// S=4
  for(q=0; q<Q; q++){// Q =9
    OA[q] += IA[q+s] * W[s];
  }
```



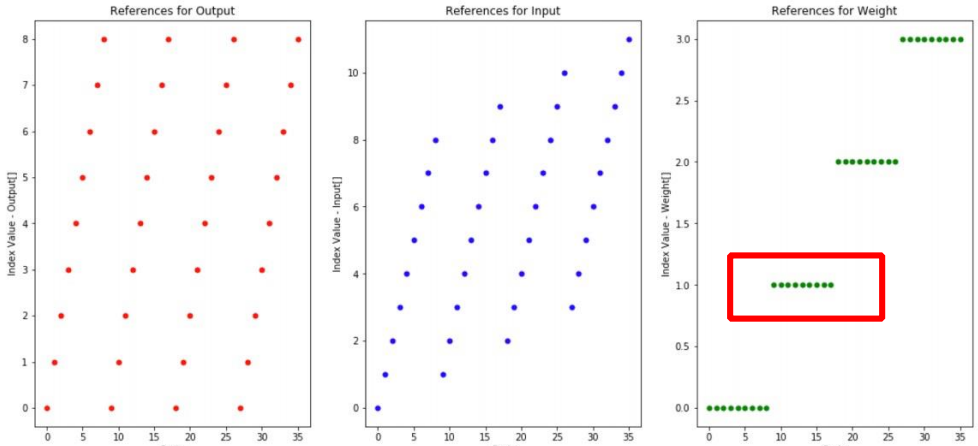
Eyeriss Tutorial



Buffer Access Pattern (WS)

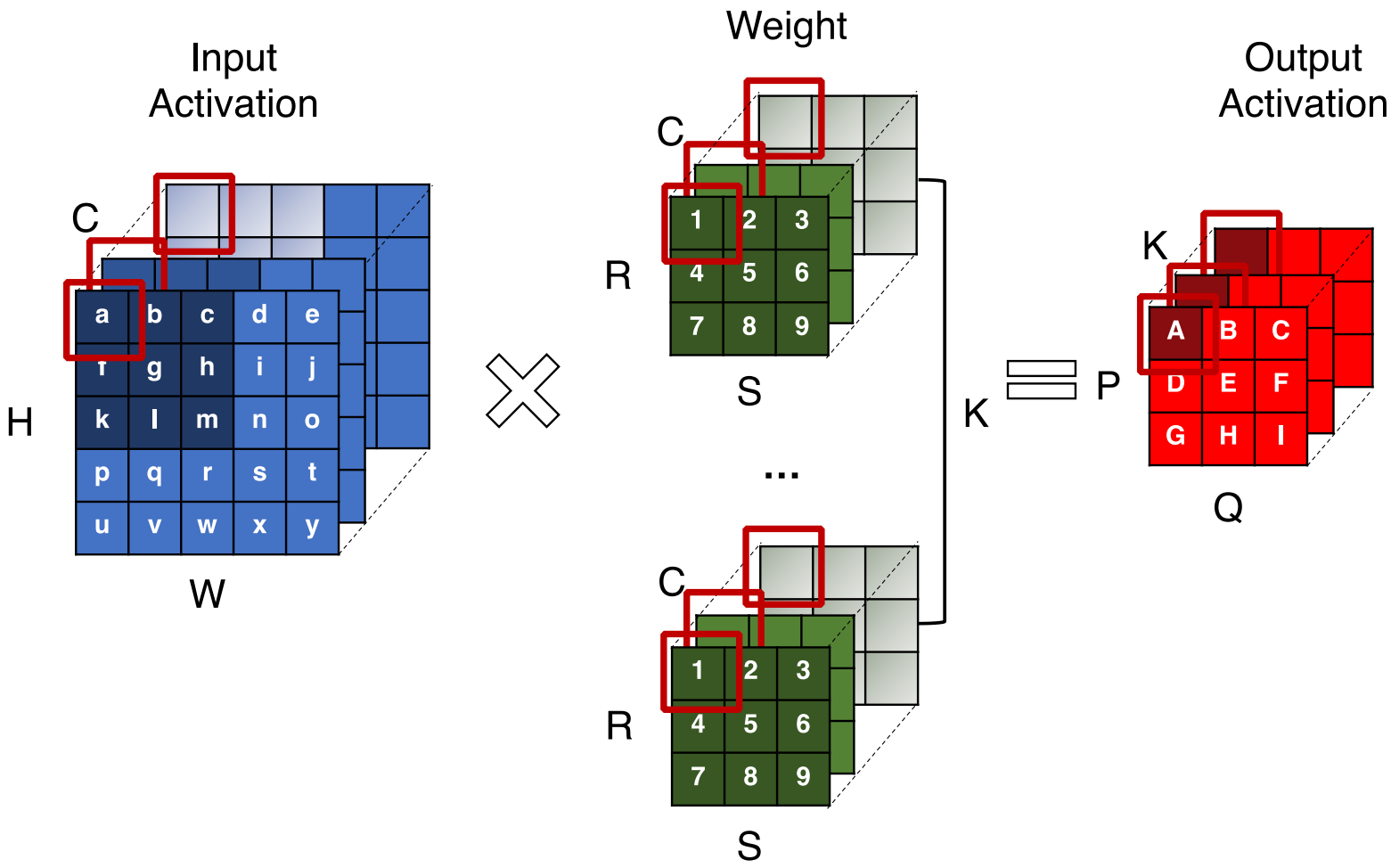



```
for (s=0; s<S; s++){// S=4
  for(q=0; q<Q; q++){// Q =9
    OA[q] += IA[q+s] * W[s];
  }
}
```

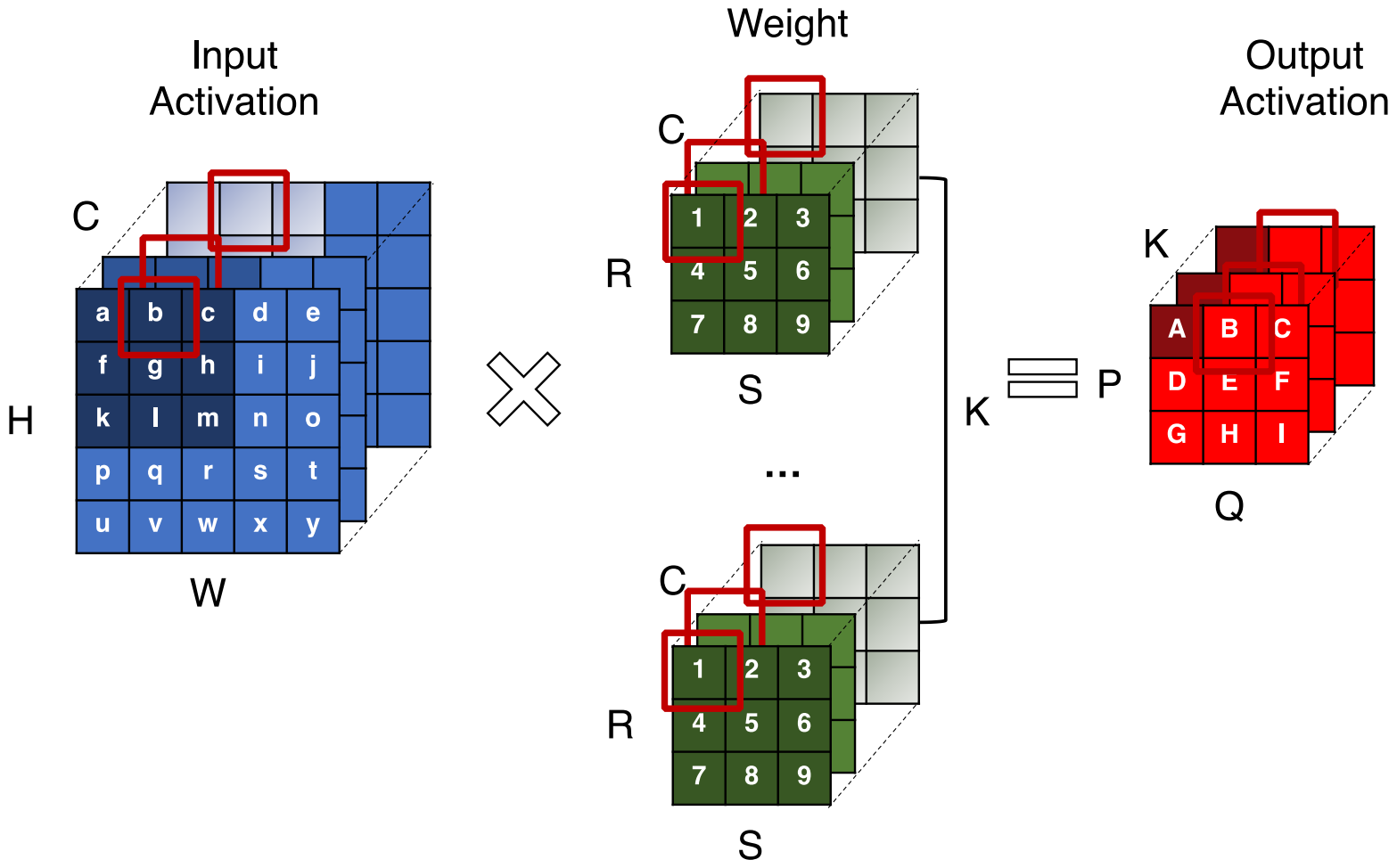


Observation: Single weight is reused Q times.

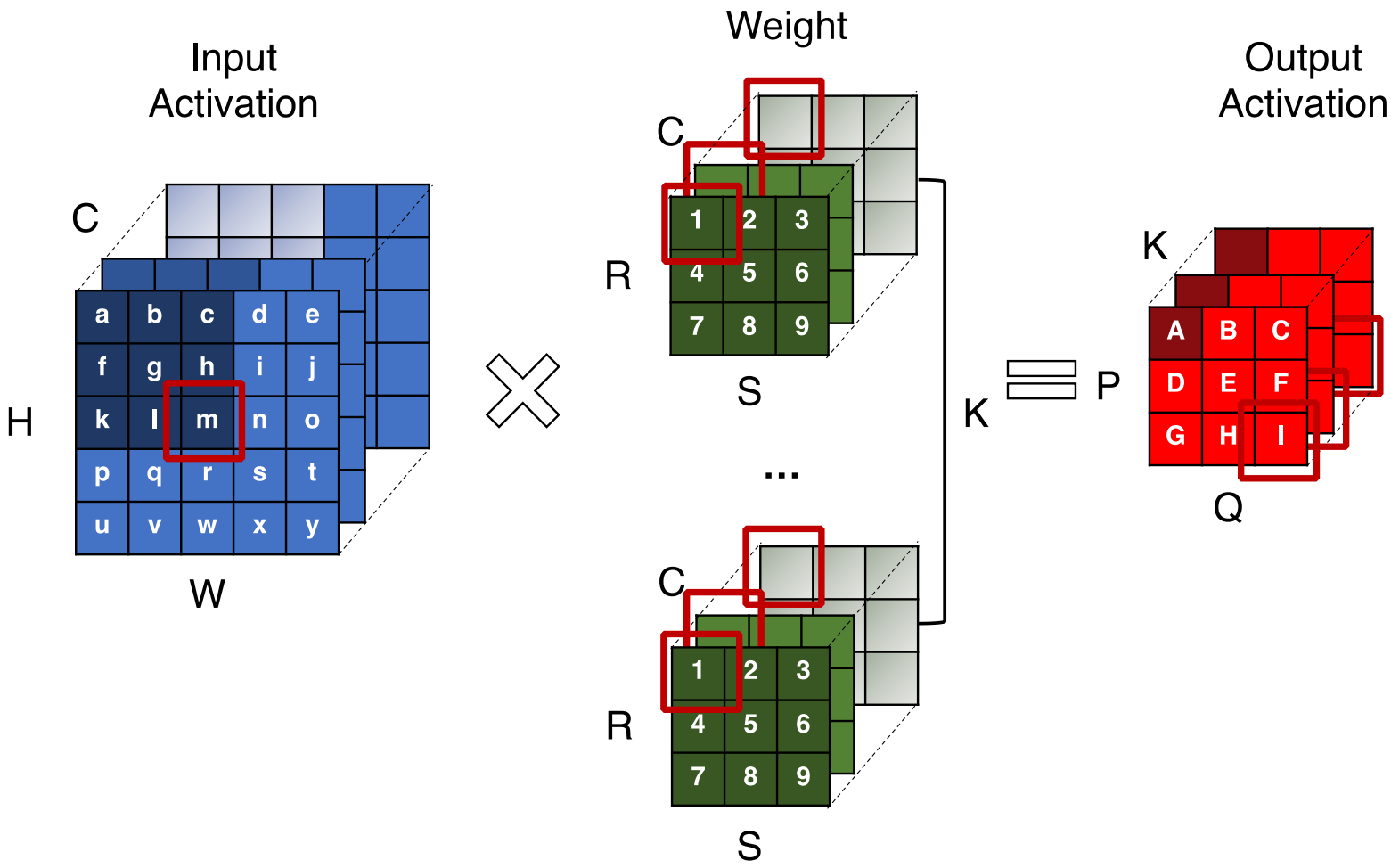
Weight Stationary Dataflow



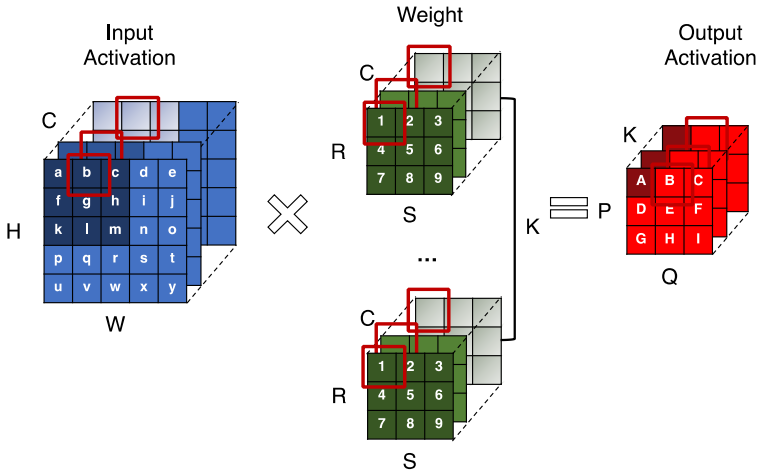
Weight Stationary Dataflow



Weight Stationary Dataflow



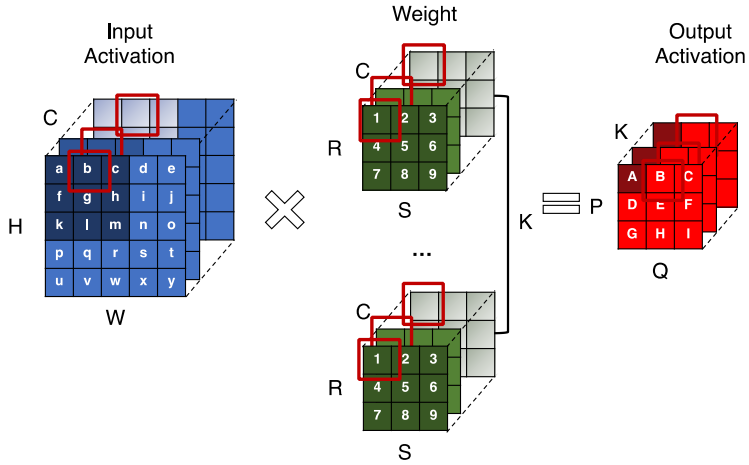
Weight Stationary Dataflow



- What we had before:

```
for (n=0; n<N; n++) {
  for (k=0; k<K; k++) {
    for (p=0; p<P; p++) {
      for (q=0; q<Q; q++) {
        OA[n][k][p][q]= 0;
        for (r=0; r<R; r++) {
          for (s=0; s<S; s++) {
            for (c=0; c<C; c++) {
              h = p * stride - pad + r;
              w = q * stride - pad + s;
              OA[n][k][p][q] +=
                        IA[n][c][h][w]
                           * W[k][c][r][s];
            }
          }
        }
      }
    }
  }
}
```

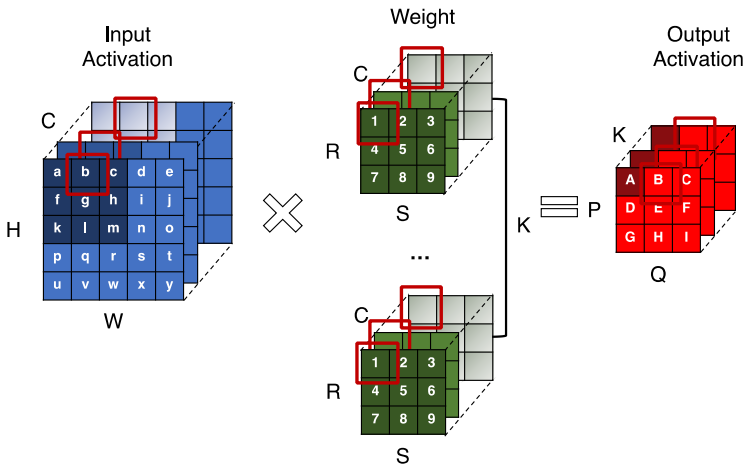
Weight Stationary Dataflow



- Change temporal ordering

```
for (n=0; n<N; n++) {
  for (r=0; r<R; r++) {
    for (s=0; s<S; s++) {
      for (c=0; c<C; c++) {
        for (k=0; k<K; k++) {
          float curr_w = W[r][s][c][k];
          for (p=0; p<P; p++) {
            for (q=0; q<Q; q++) {
              h = p * stride - pad + r;
              w = q * stride - pad + s;
              OA[n][k][p][q] +=
                        IA[n][c][h][w]
                           * curr_w;
            }
          }
        }
      }
    }
  }
}
```

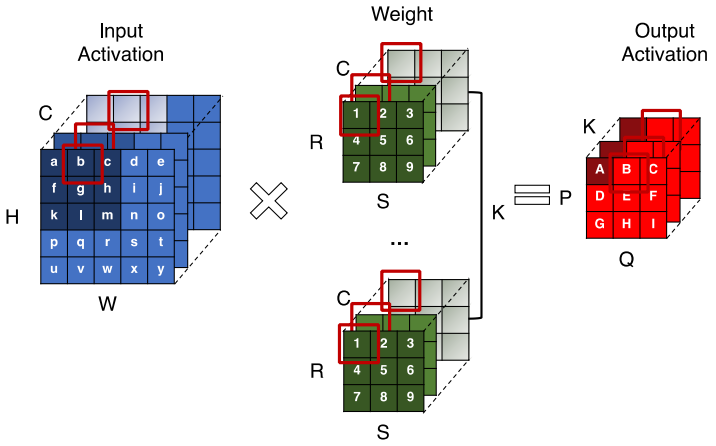
Weight Stationary Dataflow



- Apply spatial parallelism

```
for (n=0; n<N; n++) {
  for (r=0; r<R; r++) {
    for (s=0; s<S; s++) {
      spatial_for (c=0; c<C; c++) {
        spatial_for (k=0; k<K; k++) {
          float curr_w = W[r][s][c][k];
          for (p=0; p<P; p++) {
            for (q=0; q<Q; q++) {
              h = p * stride - pad + r;
              w = q * stride - pad + s;
              OA[n][k][p][q] +=
                    IA[n][c][h][w]
                       * curr_w;
            }
          }
        }
      }
    }
  }
}
```

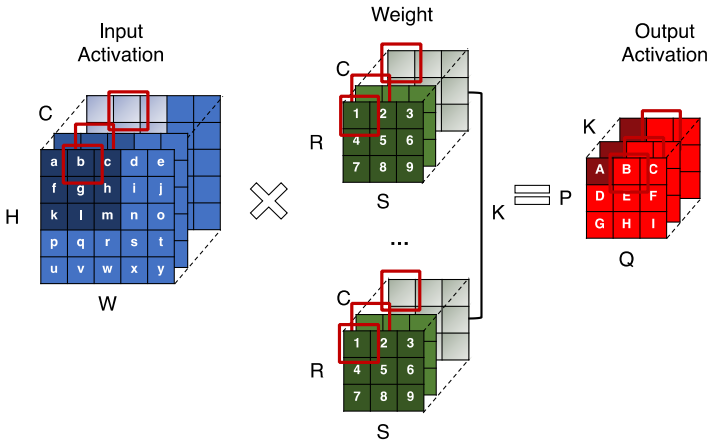
Weight Stationary Dataflow



- Apply temporal tiling

```
for (n=0; n<N; n++) {
  for (r=0; r<R; r++) {
    for (s=0; s<S; s++) {
      for (c_t=0; c_t<C/16; c_t++) {
        for (k_t=0; k_t<K/64; k_t++) {
          spatial_for (c_s=0; c_s<16; c_s++) {
            spatial_for (k_s=0; k_s<64; k_s++) {
              int curr_c = c_t * 16 + c_s;
              int curr_k = k_t * 64 + k_s;
              float curr_w = W[r][s][curr_c][curr_k];
              for (p=0; p<P; p++) {
                for (q=0; q<Q; q++) {
                  h = p * stride - pad + r;
                  w = q * stride - pad + s;
                  OA[n][curr_k][p][q] +=
                          IA[n][curr_c][h][w]
                          * curr_w;
      }}}}
    }
  }
}
```

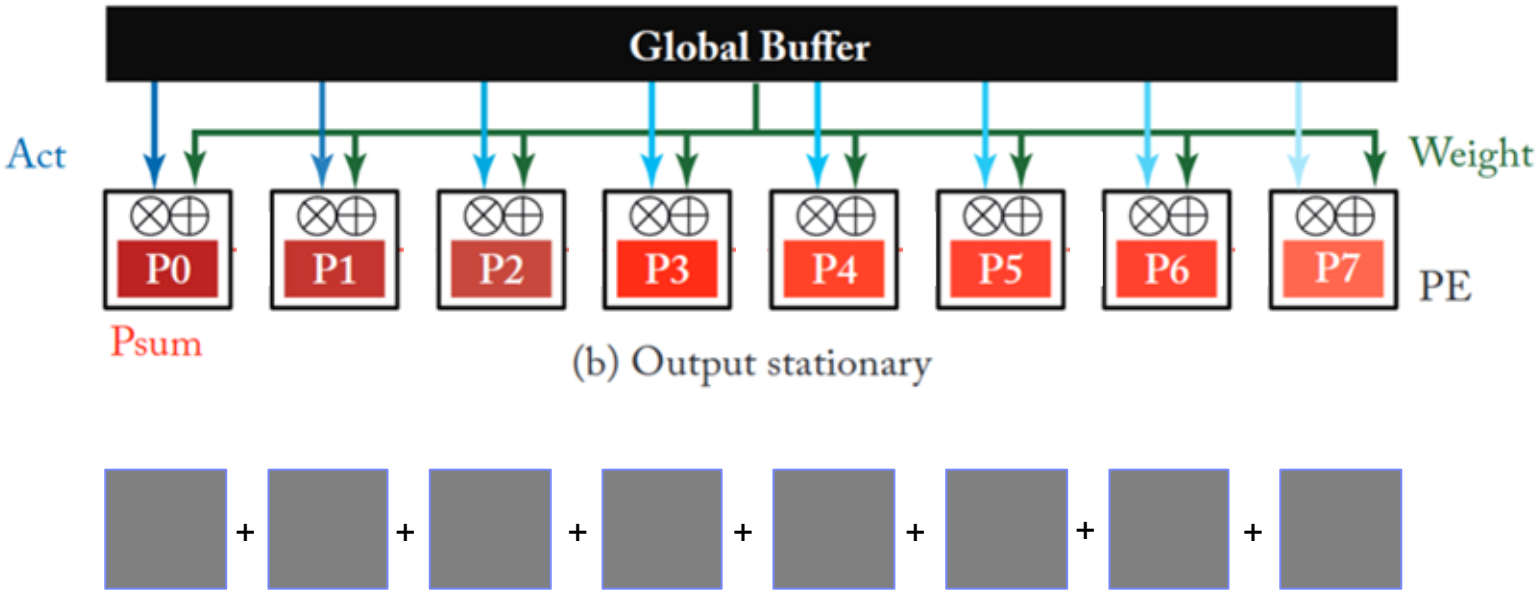
Weight Stationary Dataflow



- **NVDLA** Dataflow (nvdla.org)

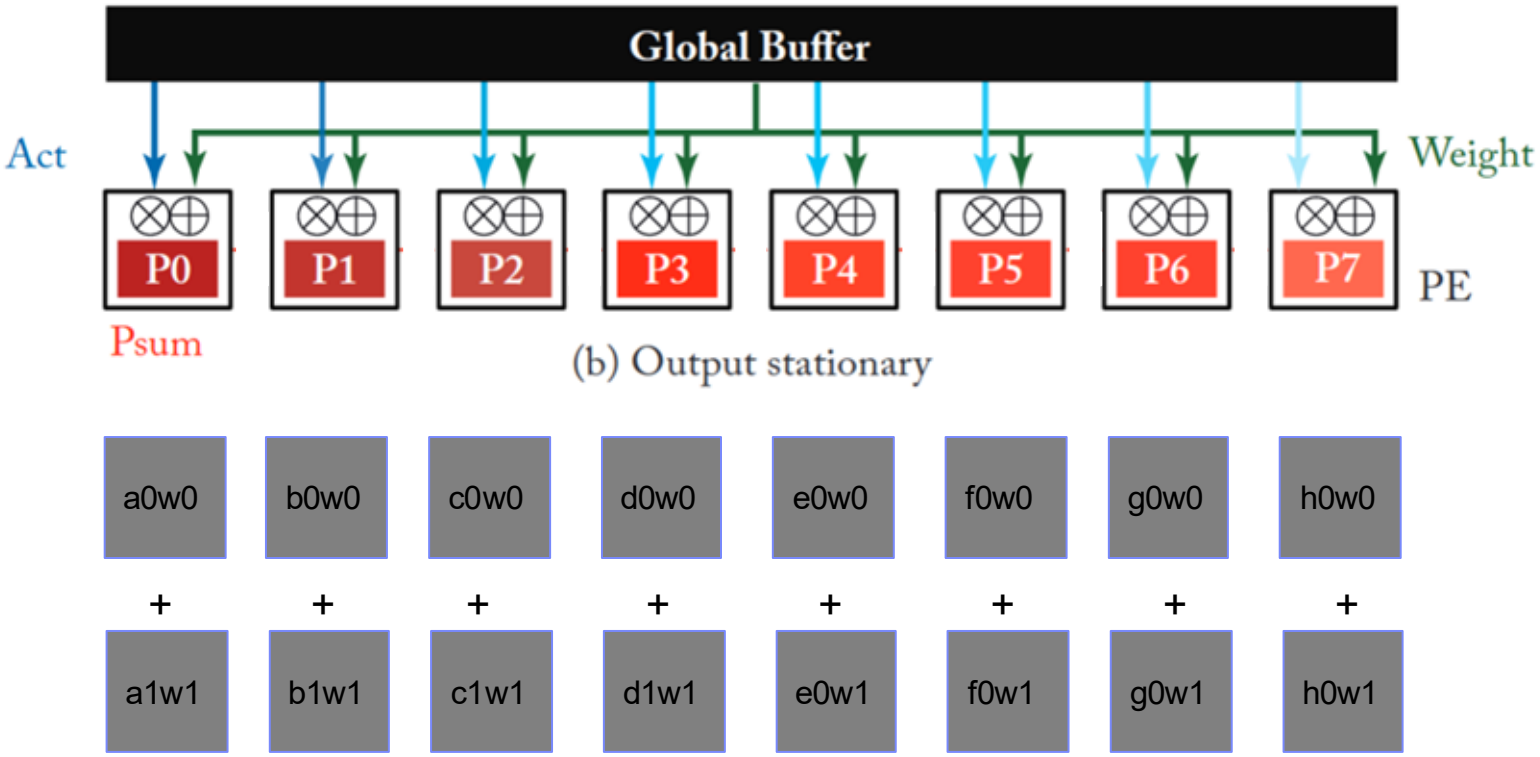
```
for (n=0; n<N; n++) {
  for (r=0; r<R; r++) {
    for (s=0; s<S; s++) {
      for (c_t=0; c_t<C/16; c_t++) {
        for (k_t=0; k_t<K/64; k_t++) {
          spatial_for (c_s=0; c_s<16; c_s++) {
            spatial_for (k_s=0; k_s<64; k_s++) {
              int curr_c = c_t * 16 + c_s;
              int curr_k = k_t * 64 + k_s;
              float curr_w = W[r][s][curr_c][curr_k];
              for (p=0; p<P; p++) {
                for (q=0; q<Q; q++) {
                  h = p * stride - pad + r;
                  w = q * stride - pad + s;
                  OA[n][curr_k][p][q] +=
                          IA[n][curr_c][h][w]
                          * curr_w;
      }}}}
    }
  }
}
```

What to Keep Stationary? Outputs



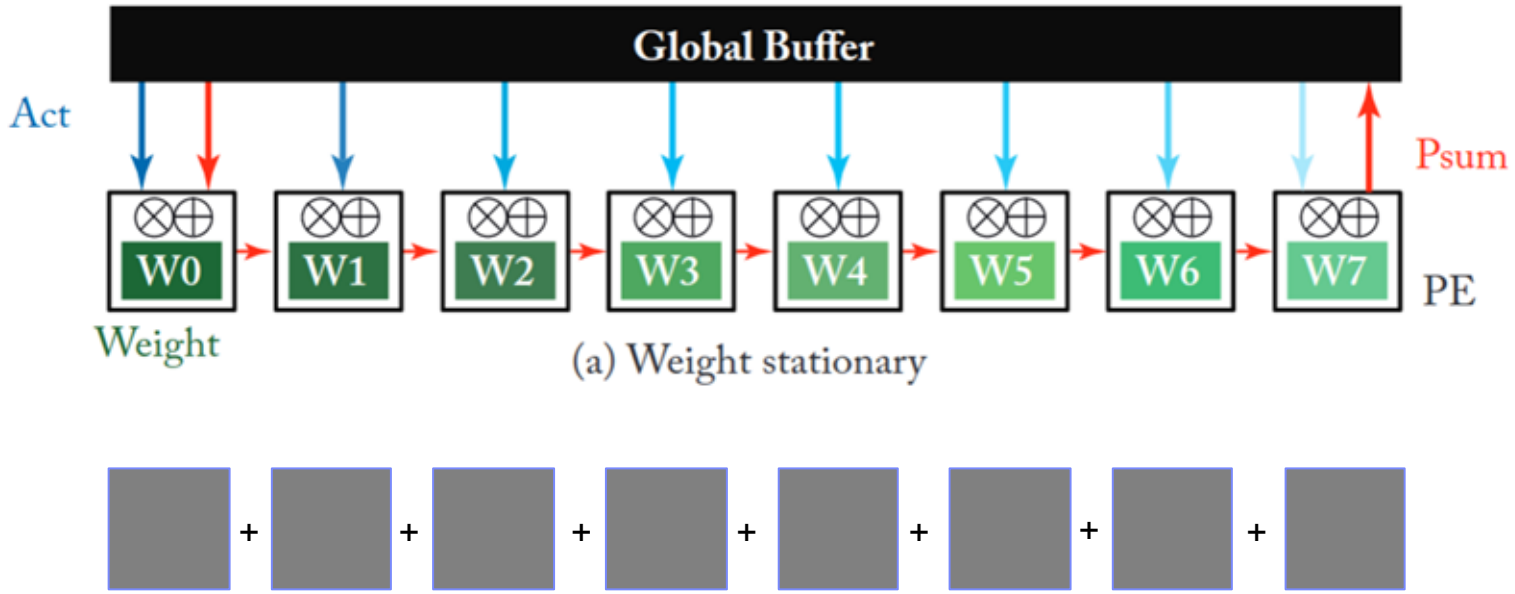
Source: V. Sze et al. "Efficient Processing of Deep Neural Networks" 2020

What to Keep Stationary? Outputs



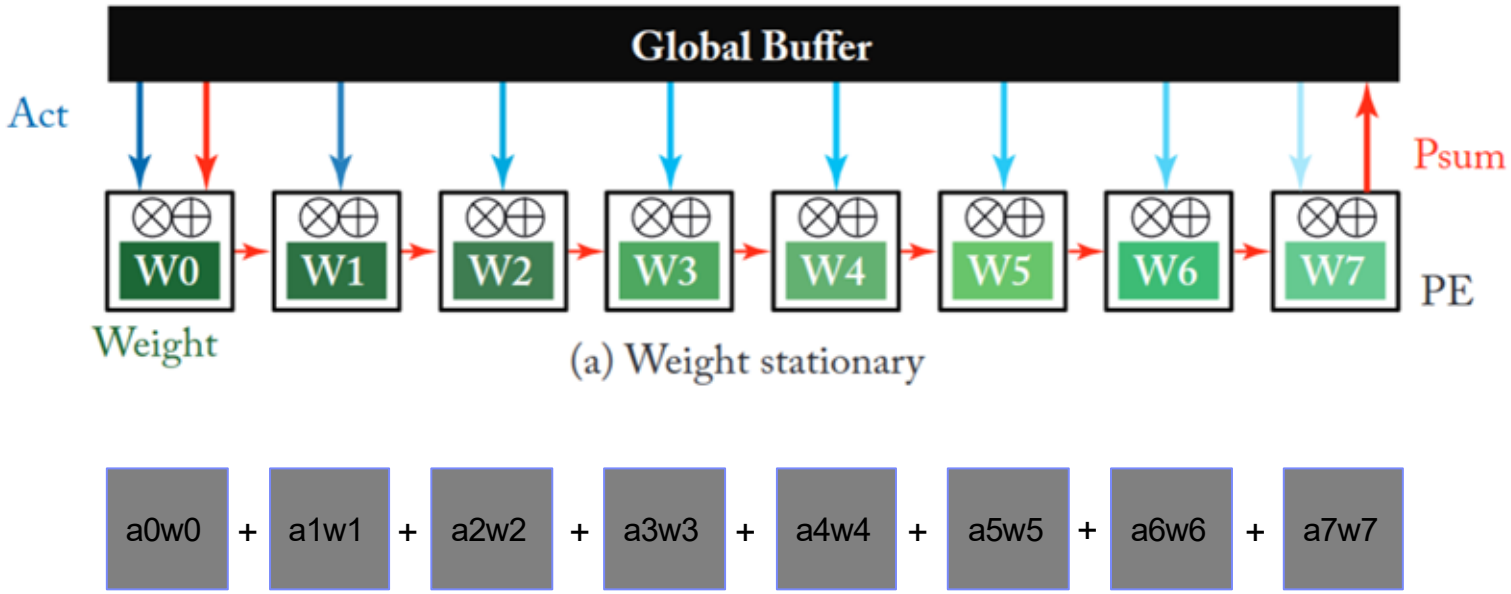
Source: V. Sze et al. "Efficient Processing of Deep Neural Networks" 2020

What to Keep Stationary? Weights



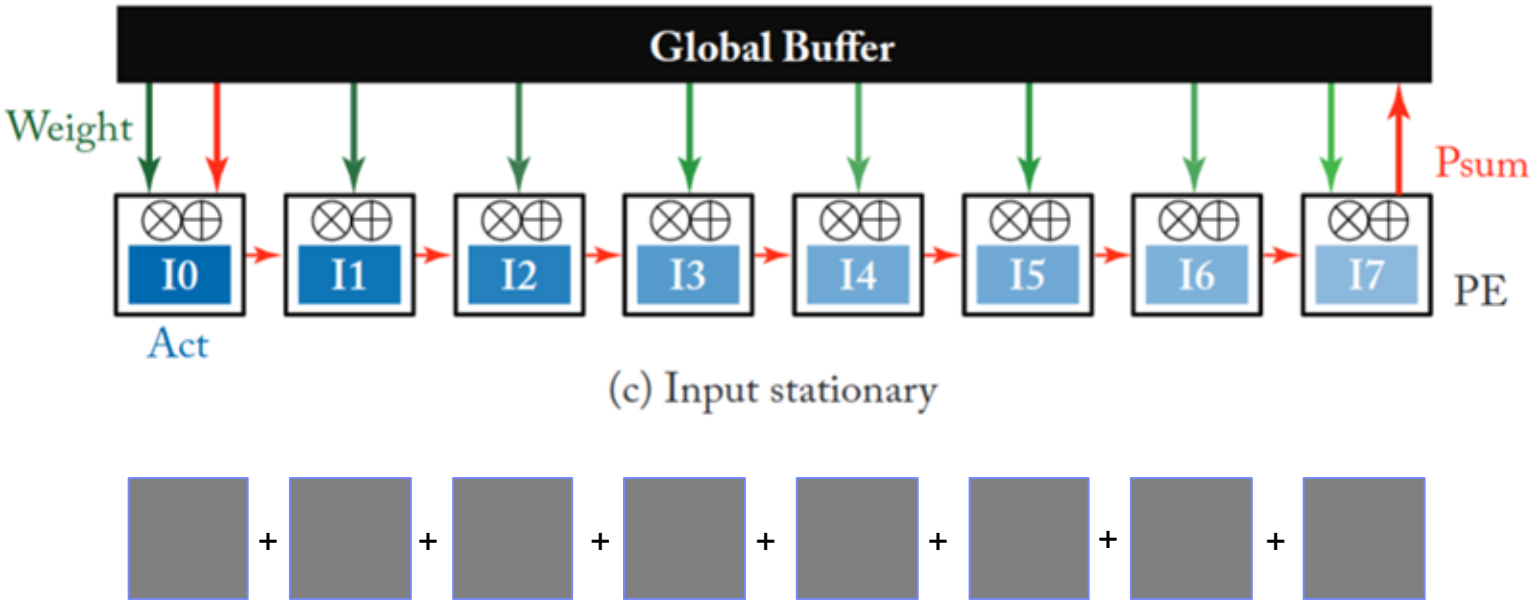
Source: V. Sze et al. "Efficient Processing of Deep Neural Networks" 2020

What to Keep Stationary? Weights



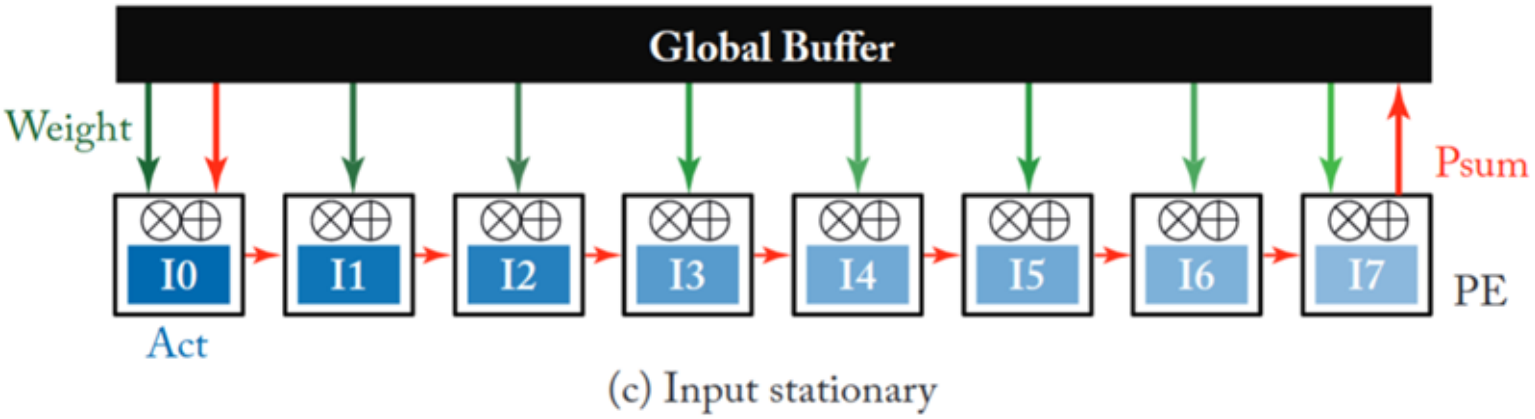
Source: V. Sze et al. "Efficient Processing of Deep Neural Networks" 2020

What to Keep Stationary? Inputs



Source: V. Sze et al. "Efficient Processing of Deep Neural Networks" 2020

What to Keep Stationary? Inputs



(c) Input stationary

a0w0 + a1w1 + a2w2 + a3w3 + a4w4 + a5w5 + a6w6 + a7w7

Source: V. Sze et al. "Efficient Processing of Deep Neural Networks" 2020